تقرير تحليل ومناقشة نتائج خوارزميات الترتيب

اختبار الأداء والتطبيق العملي للبيانات بحجم $N = 100,000$ عنصر (Microseconds)

أ. الخوارزميات ذات التعقيد التربيعي $O(N^2)$

- **Bubble Sort (الترتيب الفقاعي):**

أظهرت أداءً سيئاً وزمن تنفيذ مرتفعاً جداً يزداد بشكل حاد مع كبر حجم البيانات في الحالات العشوائية والعكسية والجزئية، بينما سجلت أداءً سريعاً جداً في حالة البيانات المرتبة تماماً بفضل خاصية التوقف المبكر.

- **Selection Sort (الترتيب بالمقارنة المباشرة):**

أظهرت أزمنة تنفيذ متقاربة وبطيئة في جميع الحالات، لأن عدد المقارنات يظل ثابتاً بصرف النظر عن حالة الترتيب الأولي للمصفوفة.

- **Insertion Sort (الترتيب الإدراجي):**

أظهرت بطئاً في الحالات العشوائية والعكسية، لكنها حققت أفضل أداء على الإطلاق في حالة البيانات المرتبة تماماً ($143 \mu s$) بفضل التعقيد الزمني الخطي $O(N)$ في الحالة الأفضل.

ب. الخوارزميات ذات التعقيد اللوغاريتمي $O(N \log N)$

- **Merge Sort (الترتيب بالدمج):**

حققت كفاءة عالية وأداءً ثابتاً في جميع حالات البيانات، نظرًا لضمان تعقيدها الزمني $O(N \log N)$ دائماً باستخدام أسلوب "فرق تسد".

- **Quick Sort (الترتيب السريع):**

حققت أفضل أداء إجمالي وأسرع زمن تنفيذ في البيانات العشوائية والعكسية؛ وذلك بفضل استخدام المحور العشوائي (Randomized Pivot) الذي منع حدوث الحالة الأسوأ $O(N^2)$ وتجنب طفح المكدس (StackOverflowError).

ج. الاستنتاج الرئيسي والتوصيات (Conclusion)

- تثبت النتائج الفارق الشاسع بين الخوارزميات ذات التعقيد $O(N^2)$ والإنتاجية العالية للخوارزميات ذات التعقيد $O(N \log N)$ عند التعامل مع أحجام بيانات كبيرة.
- يُعد اختيار المحور العشوائي حاسماً لضمان استقرار أداء **Quick Sort** ومنع تدهوره.
- تظل **Insertion Sort** الخيار الأمثل للمصفوفات المرتبة جزئياً أو كلياً، بينما تعتبر **Quick Sort** و **Merge Sort** الخيار الأفضل للبيانات العامة والكبيرة.